

MODULE 3

PL/SQL PROGRAMMING

Exercise 1: Control Structures

```
-- Exercise 1 - control structures
-----

-- scenario 1 - give senior citizens (60+) 1% off their loan interest
declare
begin
    for c in (select CustomerID, DOB from Customers) loop
        if trunc(months_between(sysdate, c.DOB)/12) > 60 then
            update Loans set InterestRate = InterestRate * 0.99 where CustomerID = c.CustomerID;
        end if;
    end loop;
    commit;
end;
/

-- scenario 2 - mark customers VIP if balance > 10000
BEGIN
    FOR c IN (SELECT CustomerID, Balance FROM Customers) LOOP
        IF c.Balance > 10000 THEN
            UPDATE Customers SET IsVIP = 'Y' WHERE CustomerID = c.CustomerID;
        END IF;
    END LOOP;
    COMMIT;
END;
/

-- scenario 3 - loans due in the next 30 days, print reminder
DECLARE
    CURSOR c1 IS
        SELECT l.LoanID, l.EndDate, cu.Name
        FROM Loans l, Customers cu
        WHERE l.CustomerID = cu.CustomerID
        AND l.EndDate BETWEEN SYSDATE AND SYSDATE+30;
BEGIN
    FOR r IN c1 LOOP
        DBMS_OUTPUT.PUT_LINE('Reminder: Loan ' || r.LoanID || ' for ' || r.Name || ' due on ' || r.EndDate);
    END LOOP;
END;
/
```

OUTPUT:

```

SQL> declare
  begin
    for c in (select CustomerID, DOB from Customers) loop
      if trunc(months_between(sysdate, c.DOB)/12) > 60 then...
Show more...

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.010

SQL> BEGIN
  FOR c IN (SELECT CustomerID, Balance FROM Customers) LOOP
    IF c.Balance > 10000 THEN
      UPDATE Customers SET IsVIP = 'Y' WHERE CustomerID = c.CustomerID;...
Show more...

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.007

SQL> DECLARE
  CURSOR c1 IS
    SELECT l.LoanID, l.EndDate, cu.Name
    FROM Loans l, Customers cu...
Show more...

Reminder: Loan 2 for Robert King due 23-JUL-26

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.014

```

Exercise 2: Error Handling

```

1  -- Exercise 2 - error handling
2  -----
3
4  CREATE OR REPLACE PROCEDURE SafeTransferFunds(p_from NUMBER, p_to NUMBER, p_amt NUMBER)
5  IS
6    bal NUMBER;
7  BEGIN
8    SELECT Balance INTO bal FROM Accounts WHERE AccountID = p_from;
9
10   IF bal < p_amt THEN
11     dbms_output.put_line('Not enough funds in account ' || p_from);
12   ELSE
13     UPDATE Accounts SET Balance = Balance - p_amt WHERE AccountID = p_from;
14     UPDATE Accounts SET Balance = Balance + p_amt WHERE AccountID = p_to;
15     INSERT INTO Transactions VALUES (trans_seq.NEXTVAL, p_from, SYSDATE, p_amt, 'Withdrawal');
16     INSERT INTO Transactions VALUES (trans_seq.NEXTVAL, p_to, SYSDATE, p_amt, 'Deposit');
17     COMMIT;
18   END IF;
19 EXCEPTION
20   WHEN OTHERS THEN
21     ROLLBACK;
22     dbms_output.put_line('something went wrong: ' || SQLERRM);
23 END;
24 /
25
26 CREATE OR REPLACE PROCEDURE UpdateSalary(p_id NUMBER, p_pct NUMBER)
27 IS
28   v_count NUMBER;
29 BEGIN
30   SELECT COUNT(*) INTO v_count FROM Employees WHERE EmployeeID = p_id;
31   IF v_count = 0 THEN
32     dbms_output.put_line('Employee ' || p_id || ' not found');
33   ELSE
34     UPDATE Employees SET Salary = Salary + Salary*p_pct/100 WHERE EmployeeID = p_id;
35     COMMIT;
36   END IF;
37 END;
38 /

```

```

39
40 CREATE OR REPLACE PROCEDURE AddNewCustomer(p_id NUMBER, p_name VARCHAR2, p_dob DATE, p_bal NUMBER)
41 IS
42 BEGIN
43     INSERT INTO Customers(CustomerID, Name, DOB, Balance, LastModified)
44     VALUES (p_id, p_name, p_dob, p_bal, SYSDATE);
45     COMMIT;
46 EXCEPTION
47     WHEN DUP_VAL_ON_INDEX THEN
48         dbms_output.put_line('Customer ' || p_id || ' already exists, cannot add');
49 END;
50 /

```

OUTPUT:

```

SQL> CREATE OR REPLACE PROCEDURE SafeTransferFunds(p_from NUMBER, p_to NUMBER, p_amt NUMBER)
      IS
          bal NUMBER;
      BEGIN...
Show more...

```

Procedure SAFETRANSFERFUNDS compiled

Elapsed: 00:00:00.010

```

SQL> CREATE OR REPLACE PROCEDURE UpdateSalary(p_id NUMBER, p_pct NUMBER)
      IS
          v_count NUMBER;
      BEGIN...
Show more...

```

Procedure UPDATESALARY compiled

Elapsed: 00:00:00.007

```

SQL> CREATE OR REPLACE PROCEDURE AddNewCustomer(p_id NUMBER, p_name VARCHAR2, p_dob DATE, p_bal NUMBER)
      IS
      BEGIN
          INSERT INTO Customers(CustomerID, Name, DOB, Balance, LastModified)...
Show more...

```

Procedure ADDNEWCUSTOMER compiled

Elapsed: 00:00:00.005

Exercise 3: Stored Procedures

CODE:

```

1 CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS
2 BEGIN
3     UPDATE Accounts SET Balance = Balance + Balance*0.01 WHERE AccountType = 'Savings';
4     COMMIT;
5 END;
6 /
7
8 CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(p_dept VARCHAR2, p_bonus NUMBER) IS
9 BEGIN
10    UPDATE Employees SET Salary = Salary + Salary*p_bonus/100 WHERE Department = p_dept;
11    COMMIT;
12 END;
13 /
14
15 CREATE OR REPLACE PROCEDURE TransferFunds(p_from NUMBER, p_to NUMBER, p_amt NUMBER) IS
16     bal NUMBER;
17 BEGIN
18     SELECT Balance INTO bal FROM Accounts WHERE AccountID = p_from;
19     IF bal >= p_amt THEN
20         UPDATE Accounts SET Balance = Balance - p_amt WHERE AccountID = p_from;
21         UPDATE Accounts SET Balance = Balance + p_amt WHERE AccountID = p_to;
22     COMMIT;
23     ELSE
24         dbms_output.put_line('cant transfer, not enough balance');
25     END IF;
26 END;
27 /

```

OUTPUT:

```

SQL> CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS
      BEGIN
          UPDATE Accounts SET Balance = Balance + Balance*0.01 WHERE AccountType = 'Savings';
          COMMIT;...
Show more...

```

Procedure PROCESSMONTHLYINTEREST compiled

Elapsed: 00:00:00.004

```

SQL> CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(p_dept VARCHAR2, p_bonus NUMBER) IS
      BEGIN
          UPDATE Employees SET Salary = Salary + Salary*p_bonus/100 WHERE Department = p_dept;
          COMMIT;...
Show more...

```

Procedure UPDATEEMPLOYEEBONUS compiled

Elapsed: 00:00:00.002

```

SQL> CREATE OR REPLACE PROCEDURE TransferFunds(p_from NUMBER, p_to NUMBER, p_amt NUMBER) IS
      bal NUMBER;
      BEGIN
          SELECT Balance INTO bal FROM Accounts WHERE AccountID = p_from;...
Show more...

```

Procedure TRANSFERFUNDS compiled

Elapsed: 00:00:00.003

Exercise 4: Functions

```

1 CREATE OR REPLACE FUNCTION CalculateAge(p_dob DATE) RETURN NUMBER IS
2 BEGIN
3     RETURN trunc(months_between(sysdate, p_dob)/12);
4 END;
5 /
6
7 CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(p_amt NUMBER, p_rate NUMBER, p_years NUMBER)
8 RETURN NUMBER IS
9     r NUMBER := (p_rate/100)/12;
10    n NUMBER := p_years*12;
11 BEGIN
12     RETURN round(p_amt*r / (1 - power(1+r, -n)), 2);
13 END;
14 /
15
16 CREATE OR REPLACE FUNCTION HasSufficientBalance(p_acc NUMBER, p_amt NUMBER) RETURN VARCHAR2 IS
17     bal NUMBER;
18 BEGIN
19     SELECT Balance INTO bal FROM Accounts WHERE AccountID = p_acc;
20     IF bal >= p_amt THEN
21         RETURN 'Y';
22     ELSE
23         RETURN 'N';
24     END IF;
25 END;
26 /
27

```

Output:

```

SQL> CREATE OR REPLACE FUNCTION CalculateAge(p_dob DATE) RETURN NUMBER IS
      BEGIN
      RETURN trunc(months_between(sysdate, p_dob)/12);
      END;

Function CALCULATEAGE compiled

Elapsed: 00:00:00.025

SQL> CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(p_amt NUMBER, p_rate NUMBER, p_years NUMBER)
      RETURN NUMBER IS
      r NUMBER := (p_rate/100)/12;
      n NUMBER := p_years*12;...
Show more...

Function CALCULATEMONTHLYINSTALLMENT compiled

Elapsed: 00:00:00.012

SQL> CREATE OR REPLACE FUNCTION HasSufficientBalance(p_acc NUMBER, p_amt NUMBER) RETURN VARCHAR2 IS
      bal NUMBER;
      BEGIN
      SELECT Balance INTO bal FROM Accounts WHERE AccountID = p_acc;...
Show more...

Function HASSUFFICIENTBALANCE compiled

Elapsed: 00:00:00.016

```

Exercise 5: Triggers

```

1 CREATE OR REPLACE TRIGGER trg_CustomerLastModified
2 BEFORE UPDATE ON Customers
3 FOR EACH ROW
4 BEGIN
5     :NEW.LastModified := SYSDATE;
6 END;
7 /
8
9 CREATE OR REPLACE TRIGGER trg_LogTransaction
10 AFTER INSERT ON Transactions
11 FOR EACH ROW
12 BEGIN
13     INSERT INTO AuditLog(TransactionID, AccountID, Amount, TransactionType)
14     VALUES (:NEW.TransactionID, :NEW.AccountID, :NEW.Amount, :NEW.TransactionType);
15 END;
16 /
17
18 CREATE OR REPLACE TRIGGER trg_CheckTransactionRules
19 BEFORE INSERT ON Transactions
20 FOR EACH ROW
21 DECLARE
22     bal NUMBER;
23 BEGIN
24     IF :NEW.TransactionType = 'Deposit' AND :NEW.Amount <= 0 THEN
25         RAISE_APPLICATION_ERROR(-20001, 'deposit must be positive');
26     END IF;
27
28     IF :NEW.TransactionType = 'Withdrawal' THEN
29         SELECT Balance INTO bal FROM Accounts WHERE AccountID = :NEW.AccountID;
30         IF :NEW.Amount > bal THEN
31             RAISE_APPLICATION_ERROR(-20002, 'withdrawal more than balance');
32         END IF;
33     END IF;
34 END;
35 /

```

OUTPUT:

```

SQL> CREATE OR REPLACE TRIGGER trg_CustomerLastModified
      BEFORE UPDATE ON Customers
      FOR EACH ROW
      BEGIN...
Show more...

```

Trigger TRG_CUSTOMERLASTMODIFIED compiled

Elapsed: 00:00:00.025

```

SQL> CREATE OR REPLACE TRIGGER trg_LogTransaction
      AFTER INSERT ON Transactions
      FOR EACH ROW
      BEGIN...
Show more...

```

Trigger TRG_LOGTRANSACTION compiled

Elapsed: 00:00:00.015

```
SQL> CREATE OR REPLACE TRIGGER trg_CheckTransactionRules
  BEFORE INSERT ON Transactions
  FOR EACH ROW
  DECLARE...
Show more...
```

Trigger TRG_CHECKTRANSACTIONRULES compiled

Elapsed: 00:00:00.016

Exercise 6: Cursors

```
1  CREATE OR REPLACE PROCEDURE GenerateMonthlyStatements IS
2      CURSOR c1 IS
3          SELECT cu.Name, t.TransactionID, t.Amount, t.TransactionType
4              FROM Transactions t, Accounts a, Customers cu
5              WHERE t.AccountID = a.AccountID AND a.CustomerID = cu.CustomerID
6                  AND EXTRACT(MONTH FROM t.TransactionDate) = EXTRACT(MONTH FROM SYSDATE);
7      v_name VARCHAR2(100);
8      v_id NUMBER;
9      v_amt NUMBER;
10     v_type VARCHAR2(10);
11 BEGIN
12     OPEN c1;
13     LOOP
14         FETCH c1 INTO v_name, v_id, v_amt, v_type;
15         EXIT WHEN c1%NOTFOUND;
16         dbms_output.put_line(v_name || ' - txn ' || v_id || ' ' || v_type || ' $' || v_amt);
17     END LOOP;
18     CLOSE c1;
19 END;
20 /
21
22 -- SCENARIO 2 --
23 CREATE OR REPLACE PROCEDURE ApplyAnnualFee(p_fee NUMBER) IS
24     CURSOR c1 IS SELECT AccountID FROM Accounts;
25     v_acc NUMBER;
26 BEGIN
27     OPEN c1;
28     LOOP
29         FETCH c1 INTO v_acc;
30         EXIT WHEN c1%NOTFOUND;
31         UPDATE Accounts SET Balance = Balance - p_fee WHERE AccountID = v_acc;
32     END LOOP;
33     CLOSE c1;
34     COMMIT;
35 END;
36 /
37
38 --SCENARIO 3
39 CREATE OR REPLACE PROCEDURE UpdateLoanInterestRates(p_pct NUMBER) IS
40     CURSOR c1 IS SELECT LoanID FROM Loans;
41     v_id NUMBER;
42 BEGIN
43     OPEN c1;
44     LOOP
45         FETCH c1 INTO v_id;
46         EXIT WHEN c1%NOTFOUND;
47         UPDATE Loans SET InterestRate = InterestRate + InterestRate*p_pct/100 WHERE LoanID = v_id;
48     END LOOP;
49     CLOSE c1;
50     COMMIT;
51 END;
52 /
```

OUTPUT:

```
SQL> CREATE OR REPLACE PROCEDURE GenerateMonthlyStatements IS
    CURSOR c1 IS
        SELECT cu.Name, t.TransactionID, t.Amount, t.TransactionType
        FROM Transactions t, Accounts a, Customers cu...
Show more...
```

Procedure GENERATEMONTHLYSTATEMENTS compiled

Elapsed: 00:00:00.005

```
SQL> CREATE OR REPLACE PROCEDURE ApplyAnnualFee(p_fee NUMBER) IS
    CURSOR c1 IS SELECT AccountID FROM Accounts;
    v_acc NUMBER;
    BEGIN...
Show more...
```

Procedure APPLYANNUALFEE compiled

Elapsed: 00:00:00.004

```
SQL> CREATE OR REPLACE PROCEDURE UpdateLoanInterestRates(p_pct NUMBER) IS
    CURSOR c1 IS SELECT LoanID FROM Loans;
    v_id NUMBER;
    BEGIN...
Show more...
```

Procedure UPDATELOANINTERESTRATES compiled

Elapsed: 00:00:00.003

Exercise 7: Packages

```
-- exercise 7 packages
-- package 1 - customer stuff
CREATE OR REPLACE PACKAGE CustomerManagement IS
    PROCEDURE AddCustomer(p_id NUMBER, p_name VARCHAR2, p_dob DATE, p_bal NUMBER);
    PROCEDURE UpdateCustomer(p_id NUMBER, p_name VARCHAR2, p_bal NUMBER);
    FUNCTION GetBalance(p_id NUMBER) RETURN NUMBER;
END;
/
CREATE OR REPLACE PACKAGE BODY CustomerManagement IS
    PROCEDURE AddCustomer(p_id NUMBER, p_name VARCHAR2, p_dob DATE, p_bal NUMBER) IS
    BEGIN
        INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified) VALUES (p_id, p_name, p_dob, p_bal, SYSDATE);
        COMMIT;
    END;
    PROCEDURE UpdateCustomer(p_id NUMBER, p_name VARCHAR2, p_bal NUMBER) IS
    BEGIN
        UPDATE Customers SET Name=p_name, Balance=p_bal WHERE CustomerID=p_id;
        COMMIT;
    END;
    FUNCTION GetBalance(p_id NUMBER) RETURN NUMBER IS
        b NUMBER;
    BEGIN
        SELECT Balance INTO b FROM Customers WHERE CustomerID=p_id;
        RETURN b;
    END;
END;
/
```

```

-- package 2 - employee stuff
CREATE OR REPLACE PACKAGE EmployeeManagement IS
  PROCEDURE HireEmployee(p_id NUMBER, p_name VARCHAR2, p_pos VARCHAR2, p_sal NUMBER, p_dept VARCHAR2);
  PROCEDURE UpdateEmployee(p_id NUMBER, p_pos VARCHAR2, p_sal NUMBER);
  FUNCTION CalcAnnualSalary(p_id NUMBER) RETURN NUMBER;
END;
/

CREATE OR REPLACE PACKAGE BODY EmployeeManagement IS
  PROCEDURE HireEmployee(p_id NUMBER, p_name VARCHAR2, p_pos VARCHAR2, p_sal NUMBER, p_dept VARCHAR2) IS
  BEGIN
    INSERT INTO Employees VALUES (p_id,p_name,p_pos,p_sal,p_dept,SYSDATE);
    COMMIT;
  END;
  PROCEDURE UpdateEmployee(p_id NUMBER, p_pos VARCHAR2, p_sal NUMBER) IS
  BEGIN
    UPDATE Employees SET Position=p_pos, Salary=p_sal WHERE EmployeeID=p_id;
    COMMIT;
  END;
  FUNCTION CalcAnnualSalary(p_id NUMBER) RETURN NUMBER IS
    s NUMBER;
  BEGIN
    SELECT Salary INTO s FROM Employees WHERE EmployeeID=p_id;
    RETURN s*12;
  END;
END;
/

-- package 3 - account stuff
CREATE OR REPLACE PACKAGE AccountOperations IS
  PROCEDURE OpenAccount(p_id NUMBER, p_cust NUMBER, p_type VARCHAR2, p_bal NUMBER);
  PROCEDURE CloseAccount(p_id NUMBER);
  FUNCTION GetTotalBalance(p_cust NUMBER) RETURN NUMBER;
END;
/

CREATE OR REPLACE PACKAGE BODY AccountOperations IS
  PROCEDURE OpenAccount(p_id NUMBER, p_cust NUMBER, p_type VARCHAR2, p_bal NUMBER) IS
  BEGIN
    INSERT INTO Accounts VALUES (p_id,p_cust,p_type,p_bal,SYSDATE);
    COMMIT;
  END;
  PROCEDURE CloseAccount(p_id NUMBER) IS
  BEGIN
    DELETE FROM Accounts WHERE AccountID=p_id;
    COMMIT;
  END;
  FUNCTION GetTotalBalance(p_cust NUMBER) RETURN NUMBER IS
    t NUMBER;
  BEGIN
    SELECT NVL(SUM(Balance),0) INTO t FROM Accounts WHERE CustomerID=p_cust;
    RETURN t;
  END;
END;
/

```

OUTPUT:

```

SQL> CREATE OR REPLACE PACKAGE CustomerManagement IS
  PROCEDURE AddCustomer(p_id NUMBER, p_name VARCHAR2, p_dob DATE, p_bal NUMBER);
  PROCEDURE UpdateCustomer(p_id NUMBER, p_name VARCHAR2, p_bal NUMBER);
  FUNCTION GetBalance(p_id NUMBER) RETURN NUMBER;...
Show more...

Package CUSTOMERMANAGEMENT compiled

Elapsed: 00:00:00.169

SQL> CREATE OR REPLACE PACKAGE BODY CustomerManagement IS
  PROCEDURE AddCustomer(p_id NUMBER, p_name VARCHAR2, p_dob DATE, p_bal NUMBER) IS
  BEGIN
    INSERT INTO Customers(CustomerID,Name,DOB,Balance,LastModified) VALUES(p_id,p_name,p_dob,p_bal,SYSDATE);...
Show more...

Package Body CUSTOMERMANAGEMENT compiled

Elapsed: 00:00:00.150

```

```
SQL> CREATE OR REPLACE PACKAGE EmployeeManagement IS
    PROCEDURE HireEmployee(p_id NUMBER, p_name VARCHAR2, p_pos VARCHAR2, p_sal NUMBER, p_dept VARCHAR2);
    PROCEDURE UpdateEmployee(p_id NUMBER, p_pos VARCHAR2, p_sal NUMBER);
    FUNCTION CalcAnnualSalary(p_id NUMBER) RETURN NUMBER;...
Show more...
```

Package EMPLOYEEMANAGEMENT compiled

Elapsed: 00:00:00.145

```
SQL> CREATE OR REPLACE PACKAGE BODY EmployeeManagement IS
    PROCEDURE HireEmployee(p_id NUMBER, p_name VARCHAR2, p_pos VARCHAR2, p_sal NUMBER, p_dept VARCHAR2) IS
    BEGIN
        INSERT INTO Employees VALUES(p_id,p_name,p_pos,p_sal,p_dept,SYSDATE);...
Show more...
```

Package Body EMPLOYEEMANAGEMENT compiled

Elapsed: 00:00:00.149

```
SQL> CREATE OR REPLACE PACKAGE AccountOperations IS
    PROCEDURE OpenAccount(p_id NUMBER, p_cust NUMBER, p_type VARCHAR2, p_bal NUMBER);
    PROCEDURE CloseAccount(p_id NUMBER);
    FUNCTION GetTotalBalance(p_cust NUMBER) RETURN NUMBER;...
Show more...
```

Package ACCOUNTOPERATIONS compiled

Elapsed: 00:00:00.147

```
SQL> CREATE OR REPLACE PACKAGE BODY AccountOperations IS
    PROCEDURE OpenAccount(p_id NUMBER, p_cust NUMBER, p_type VARCHAR2, p_bal NUMBER) IS
    BEGIN
        INSERT INTO Accounts VALUES(p_id,p_cust,p_type,p_bal,SYSDATE);...
Show more...
```

Package Body ACCOUNTOPERATIONS compiled

Elapsed: 00:00:00.013